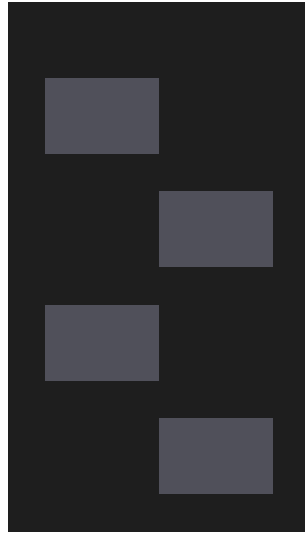




C++ - RAG Lab

Retrieval Augmented Generation

Preambulo:

Este documento contem os exercicios do modulo RAG Lab de C++.

Versao: 1.0

Sumario

I	Introducao
II	Regras gerais
III	Regras especificas do modulo
IV	Exercicio 00: Naive Retrieval
V	Exercicio 01: Token Based Retrieval
VI	Exercicio 02: TF-IDF Ranking
VII	Exercicio 03: Vector Similarity Search
VIII	Exercicio 04: Document Chunking
IX	Exercicio 05: Vector Index
X	Exercicio 06: Full RAG Pipeline
XI	Entrega e avaliacao por pares

Capitulo I

Introducao

C++ e uma linguagem de programacao de proposito geral criada por Bjarne Stroustrup como uma extensao da linguagem de programacao C, ou "C com Classes" (fonte: Wikipedia).

O objetivo deste modulo e introduzi-lo aos conceitos fundamentais de Retrieval Augmented Generation (RAG) utilizando C++. Voce implementara, passo a passo, cada componente de um sistema RAG completo: desde busca simples por palavras ate um pipeline completo de recuperacao de informacao com vetores e similaridade de cosseno.

RAG e uma tecnica moderna de IA que combina recuperacao de informacao com geracao de texto. Ao implementar cada etapa em C++, voce compreendera profundamente o funcionamento interno desses sistemas, algo que a maioria dos desenvolvedores conhece apenas pela interface de bibliotecas prontas como FAISS ou LangChain.

Seu codigo devera estar em conformidade com o padrao C++98. Estamos cientes de que o C++ moderno e bem diferente em muitos aspectos. Entao, se voce quer se tornar um desenvolvedor C++ proficiente, cabe a voce ir alem apos o 42 Common Core!

Capitulo II

Regras gerais

Compilacao

- Compile seu codigo com c++ e as flags -Wall -Wextra -Werror
- Seu codigo ainda deve compilar se voce adicionar a flag -std=c++98

Formatacao e convencoes de nomenclatura

- Os diretorios dos exercicios serao nomeados desta forma: ex00, ex01, ... , exn
- Nomeie seus arquivos, classes, funcoes, funcoes membro e atributos conforme exigido nas diretrizes.
- Escreva os nomes das classes no formato UpperCamelCase.
- A menos que especificado de outra forma, cada mensagem de saida deve terminar com um caractere de nova linha e ser exibida na saida padrao.
- Adeus Norminette! Nenhum estilo de codificacao e imposto nos modulos C++. Faca o seu melhor para escrever um codigo limpo e legivel.

Permitido/Proibido

Voce nao esta mais programando em C. E hora de C++! Portanto:

- Voce pode usar quase tudo da biblioteca padrao.
- No entanto, voce nao pode usar nenhuma outra biblioteca externa. Isso significa que bibliotecas C++11 (e formas derivadas) e Boost sao proibidas. As seguintes funcoes tambem sao proibidas: `*printf()`, `*alloc()` e `free()`.
- Observe que, a menos que explicitamente indicado de outra forma, as palavras-chave `using namespace <ns_name>` e `friend` sao proibidas. Caso contrario, sua nota sera -42.

Alguns requisitos de design

- Vazamento de memoria ocorre em C++ tambem. Quando voce aloca memoria (usando a palavra-chave `new`), voce deve evitar vazamentos de memoria.
- Do Modulo 02 ao Modulo 09, suas classes devem ser projetadas na Forma Canonica Ortodoxa, exceto quando explicitamente indicado de outra forma.

- Qualquer implementacao de funcao colocada em um arquivo de cabecalho (exceto para modelos de funcao) significa 0 para o exercicio.
- Voce deve ser capaz de usar cada um de seus cabecalhos independentemente de outros. Portanto, eles devem incluir todas as dependencias de que precisam. No entanto, voce deve evitar o problema de inclusao dupla adicionando protecoes de inclusao.

Leia-me

- Voce pode adicionar alguns arquivos adicionais se precisar (ou seja, para dividir seu codigo).
- As vezes, as diretrizes de um exercicio parecem curtas, mas os exemplos podem mostrar requisitos que nao estao explicitamente escritos nas instrucoes.
- Leia cada modulo completamente antes de comecar! Serio, faca isso.
- Por Odin, por Thor! Use seu cerebro!!!

Em relacao ao Makefile para projetos C++, as mesmas regras do C se aplicam (consulte o capitulo Norma sobre o Makefile).

Capitulo III

Regras especificas do modulo

Nao e permitido usar bibliotecas externas em nenhum exercicio deste modulo. Todo o codigo deve ser implementado do zero.

Cada exercicio deve ter um Makefile que compile os arquivos de origem com as flags -Wall, -Wextra e -Werror.

E aconselhavel ler o subject por completo antes de fazer os exercicios.

Voce deve entregar um Makefile para cada programa que compilara seus arquivos de origem para a saida necessaria com as flags -Wall, -Wextra e -Werror.

Voce deve usar c++, e seu Makefile nao deve dar relink.

Seu Makefile deve conter pelo menos as regras \$(NAME), all, clean, fclean e re.

Voce tera que implementar varias classes. Isso pode parecer tedioso, a menos que voce seja capaz de criar scripts para seu editor de texto favorito.

Voce tem uma certa liberdade para completar os exercicios. No entanto, siga as regras obrigatorias e nao seja preguiçoso. Voce perderia muitas informacoes uteis! Nao hesite em ler sobre conceitos teoricos.

Capitulo IV

Exercicio 00: Naive Retrieval

Exercice : 00
Naive Retrieval
Pasta de entrega : ex00/
Arquivos para entregar : Makefile, RAG.cpp
Funcoes nao permitidas : Bibliotecas externas, sistemas de indexacao prontos

Implementar um sistema simples de recuperacao de informacao.

O programa deve:

- Carregar um documento de texto.
- Receber uma pergunta do usuario.
- Procurar linhas que contenham palavras da pergunta.
- Retornar as linhas mais relevantes.

Esse exercicio introduz o conceito basico de retrieval, utilizado em sistemas RAG (Retrieval Augmented Generation).

Regras

- Nao usar bibliotecas externas.
- O programa deve compilar em c++98.
- O codigo deve ser claro e modular.
- A entrada deve ser lida de argv.

Funcoes permitidas

```
iostream, fstream, vector, string, map
```

Uso

```
./rag file.txt
```

Exemplo

Arquivo:

```
0 login retorna um token  
0 sistema usa PostgreSQL
```

Entrada:

```
login
```

Saida:

```
0 login retorna um token
```

Bonus: permitir multiplas queries em sequencia; ignorar maiusculas/minusculas.

Capitulo V

Exercicio 01: Token Based Retrieval

Exercice : 01
Token Based Retrieval
Pasta de entrega : ex01/
Arquivos para entregar : Makefile, RAG.cpp, Tokenizer.cpp, Tokenizer.hpp
Funcoes nao permitidas : Bibliotecas externas

Melhorar o algoritmo de busca do exercicio anterior.

O programa deve:

- Dividir a query em tokens (palavras).
- Comparar os tokens com cada linha do documento.
- Calcular um score de relevancia.
- Ordenar os resultados.

Regras

- Nao usar bibliotecas externas.
- Implementar um tokenizer proprio.
- Ordenar por score e desempatar por ordem original.

Funcoes permitidas

```
iostream, vector, map, sstream, string
```

Exemplo

Query:

```
login token
```

Documento:

```
login retorna token  
banco de dados postgres
```

Resultado esperado:

```
login retorna token
```

Score:

```
doc1 -> 2  
doc2 -> 0
```

Bonus: remover stopwords simples; suportar tokens com acentos.

Capitulo VI

Exercicio 02: TF-IDF Ranking

Exercice : 02
TF-IDF Ranking
Pasta de entrega : ex02/
Arquivos para entregar : Makefile, RAG.cpp, TFIDF.cpp, TFIDF.hpp
Funcoes nao permitidas : Bibliotecas externas

Implementar um sistema de ranking baseado em TF-IDF.

O programa deve:

1. Construir um vocabulario.
2. Calcular Term Frequency (TF).
3. Calcular Inverse Document Frequency (IDF).
4. Calcular o score final dos documentos.

Score:

```
TF = frequencia da palavra no documento
IDF = raridade da palavra no corpus

Score final = TF * IDF
```

Esse metodo e utilizado em motores de busca classicos.

Regras

- Nao usar bibliotecas externas.
- Evitar overflow e divisao por zero.
- Documentos devem ser normalizados (lowercase).

Funcoes permitidas

```
iostream, vector, map, cmath, string
```

Bonus: normalizar o score pelo tamanho do documento.

Capitulo VII

Exercicio 03: Vector Similarity Search

Exercise : 03
Vector Similarity Search
Pasta de entrega : ex03/
Arquivos para entregar : Makefile, RAG.cpp, VectorSearch.cpp, VectorSearch.hpp
Funcoes nao permitidas : Bibliotecas externas

Representar documentos como vetores numericos.

Implementar:

- Vetor de caracteristicas.
- Calculo de cosine similarity.
- Ranking dos documentos mais proximos da query.

Formula:

$$\text{cosine_similarity}(A, B) = \text{dot}(A, B) / (|A| * |B|)$$

Esse metodo e a base de vector search.

Regras

- Nao usar bibliotecas externas.
- Vetores devem ter tamanho fixo.
- Tratar vetores vazios corretamente.

Funcoes permitidas

```
iostream, vector, cmath, string
```

Fluxo:

```
query vector  
|  
comparar com doc vectors  
|  
top_k resultados
```

Bonus: aplicar normalizacao L2.

Capitulo VIII

Exercicio 04: Document Chunking

Exercice : 04
Document Chunking
Pasta de entrega : ex04/
Arquivos para entregar : Makefile, RAG.cpp, Chunker.cpp, Chunker.hpp
Funcoes nao permitidas : Bibliotecas externas

Dividir documentos grandes em chunks menores. RAG real nao usa documentos inteiros.

O programa deve:

- Carregar documentos grandes.
- Dividir em pedacos menores (chunks).
- Indexar cada pedaco separadamente.

Isso e essencial em sistemas RAG modernos.

Regras

- Cada chunk deve ter tamanho maximo definido.
- Manter a ordem original.
- Salvar metadados de origem.

Funcoes permitidas

```
iostream, fstream, vector, string
```

Exemplo:

```
2000 tokens
|
5 chunks
|
buscar apenas chunks relevantes
```

Bonus: permitir overlap entre chunks.

Capitulo IX

Exercicio 05: Vector Index

Exercice : 05
Vector Index
Pasta de entrega : ex05/
Arquivos para entregar : Makefile, RAG.cpp, VectorIndex.cpp, VectorIndex.hpp
Funcoes nao permitidas : Bibliotecas externas

Criar um indice vetorial simples.

O indice deve permitir:

- Armazenar embeddings.
- Buscar os k vetores mais proximos (top-k search).
- Retornar os documentos correspondentes.

Esse exercicio simula o funcionamento de bibliotecas como FAISS.

Regras

- O indice deve ser em memoria.
- A busca deve ser linear (brute force).
- Tratar o caso de k maior que o total de vetores.

Funcoes permitidas

```
iostream, vector, cmath, string
```

Estrutura sugerida:

```
struct VectorIndex
{
    std::vector<std::vector<float> > embeddings;
    search();
};
```

Bonus: persistencia em arquivo binario.

Capitulo X

Exercicio 06: Full RAG Pipeline

Exercise : 06
Full RAG Pipeline
Pasta de entrega : ex06/
Arquivos para entregar : Makefile, RAG.cpp, Retriever.cpp, Retriever.hpp
Funcoes nao permitidas : Bibliotecas externas

Construir um pipeline completo de Retrieval Augmented Generation.

Esse exercicio combina todos os conceitos anteriores.

Fluxo:

```
query
|
embedding
|
vector search
|
top_k documents
|
context generation
|
response
```

Regras

- Implementar classes separadas para cada etapa.
- Nao usar bibliotecas externas.
- O pipeline deve ser testavel em etapas.

Funcoes permitidas

```
iostream, vector, string
```

Bonus: ranking hibrido (TF-IDF + vector); soporte a multiplos documentos.

Capitulo XI

Entrega e avaliacao por pares

Entregue sua tarefa no seu repositorio Git como de costume. Apenas o trabalho dentro do seu repositorio sera avaliado durante a defesa. Nao hesite em verificar novamente os nomes dos seus arquivos para ter certeza de que estao corretos.

Estrutura final do repositorio:

```
cpp-rag-lab/  
|  
+-- ex00/   Naive Retrieval  
+-- ex01/   Token Based Retrieval  
+-- ex02/   TF-IDF Ranking  
+-- ex03/   Vector Similarity Search  
+-- ex04/   Document Chunking  
+-- ex05/   Vector Index  
+-- ex06/   Full RAG Pipeline
```

Voce tem liberdade para completar os exercicios da forma que preferir. No entanto, siga as regras obrigatorias. Nao seja preguiçoso!